

---

**Assignment 8****Learning goals:**

Apply the concepts and methods presented in chapter “Code Generation” of the lecture to some exercise problems.

**Presentation:**

Please prepare the solutions for informal presentation in class, and fill in the online check list in ZEUS.

**Task 1      MicroJava byte code**

---

Determine the MicroJava byte code implementing the following statement sequence. Assume that *i* is a local variable at address 2, and *g* is a global variable at address 3.

For each MJVM instruction, additionally represent its code address (starting at 0), instruction length (in bytes), and the resulting content of the expression stack at run time (for the first loop iteration).

```
g = 5;
i = 2;
while (i < 10) {
    g = g + i * i;
    i++;
}
```

**Task 2      MicroJava byte code**

---

Give the MicroJava byte code implementing the following statement sequence. Assume that *x* and *y* are local variables at addresses 0 and 1, respectively.

For each MJVM instruction, additionally represent its code address (starting at 0), instruction length (in bytes), and the resulting content of the expression stack at run time, assuming an initial value of 9 for *x*. Mark all instructions that will NOT be executed for this value of *x*.

```
if (x <= 1) y = 1;
else if (x % 2 == 0) y = x / 2;
else y = x * 3 + 1;
```

### Task 3      Operand descriptors – object fields

---

Consider the expression `d.stat.avg`, where `d` is a local variable at address 0 pointing to a `Dataset` object on the heap. The `Dataset` class has been declared as follows:

```
class Stat { int min, max, avg; }
class Dataset {
    int[] val;
    Stat stat;
}
```

- (a) Represent the operand descriptors and generated MJVM instructions of the given expression as on lecture slide 42.
- (b) Assume meaningful run-time values for addresses and content of the objects involved. Then represent instruction length (in bytes) and content of the expression stack for each MJVM instruction implementing the given expression.

### Task 4      Operand descriptors – array elements

---

Consider the expression `d.val[i+1]`, where `d` is the same local variable as in Task 3, and `i` is a local variable at address 1.

- (a) Represent the operand descriptors and generated MJVM instructions of the given expression as on lecture slide 47.
- (b) Assume meaningful run-time values for addresses and content of the objects involved. Then represent instruction length (in bytes) and content of the expression stack for each MJVM instruction implementing the given expression.